

Practica Final de Sistemas Interactivos

Curso 2025/26



Darío Martínez Kostyuk (p32makod)

David Martínez Molina (i32marmd)

3º de Ingeniería Informática, mención en Computación

28 de mayo de 2026

Contents

1	Introducción	2
2	Implementación de la interfaz	2
2.1	Organización del código	2
2.2	Decisiones de diseño	6
2.3	Internacionalización	6
2.4	Aspectos destacados de la implementación	6
2.5	Relación con los conceptos teóricos	7
2.6	Capturas de la interfaz	8
3	Tutorial o breve manual de usuario	15
3.1	Requisitos previos	15
3.2	Compilación	16
3.3	Ejecución	16
3.4	Uso de la aplicación	16
3.5	Vistas y capturas	17
4	Conclusiones	22
A	Anexos	23
A.1	Referencias de recursos	23

1 Introducción

Esta memoria documenta la segunda parte de la práctica de Sistemas Interactivos, centrada en la implementación de la interfaz gráfica a partir del prototipado presentado en la entrega anterior.

Datos del proyecto

- **Nombre del proyecto:** Practica Final de Sistemas Interactivos
- **Asignatura:** Sistemas Interactivos
- **Curso académico:** 2025/26
- **Autores:**
Darío Martínez Kostyuk (p32makod) y David Martínez Molina (i32marmd)
- **Titulación:** 3º de Ingeniería Informática, mención en Computación
- **Repositorio:** <https://github.com/000Volk000/uco-si-final>

Objetivo de la práctica

El objetivo de esta entrega es presentar una interfaz funcional, coherente con el prototipo inicial, internacionalizada al menos en dos idiomas y acompañada de una memoria técnica y de usuario que describa su implementación, uso y principales decisiones de diseño.

2 Implementación de la interfaz

En esta sección se explica cómo se ha estructurado la aplicación, qué decisiones se han tomado durante la implementación y qué partes resultan más relevantes desde el punto de vista de la interacción y el diseño.

2.1 Organización del código

La estructura del proyecto se divide en tres bloques. `App` actúa como clase central y coordina la navegación, el estado global, la internacionalización y la sesión del usuario. Las clases de interfaz representan cada pantalla concreta: `LoginFrame`, `RegisterFrame`, `MainScreen`, `SectionPanel`, `Product`, `CartFrame`, `PurchaseHistoryFrame` y `TrackingFrame`. Por último, `ProductDatabase`, `Product` y `CartItem` concentran los datos de catálogo, el producto activo y los elementos del carrito o del historial.

Esta separación permite que cada pantalla se encargue solo de su parte de la interacción, mientras que `App` mantiene el control de la navegación entre vistas con `CardLayout` y de la información compartida entre ellas. Además, la internacionalización se resuelve con `ResourceBundle`, de forma que los textos visibles cambian sin alterar la estructura de los paneles.

Estructura general

```
uco-si-final/
|-- documentacion.tex
|-- README.md
|-- src/
|   |-- Account.java
|   |-- App.java
|   |-- CartFrame.java
|   |-- CartItem.java
|   |-- LoginFrame.java
|   |-- MainScreen.java
|   |-- Product.java
|   |-- ProductDatabase.java
|   |-- PurchaseHistoryFrame.java
|   |-- RegisterFrame.java
|   |-- SearchFrame.java
|   |-- SectionPanel.java
|   |-- TrackingFrame.java
|   '-- assets/
|       |-- Account/
|       |-- background/
|       |-- bottomBar/
|       |-- bundle/
|       |-- fish/
|       |-- fishHook/
|       |-- fishingLine/
|       |-- fishingRod/
|       |-- fonts/
|       |-- hook/
|       |-- livingBait/
|       |-- Products/
|       |-- rod/
|       |-- searchBar/
|       |-- sections/
|       |-- stool/
|       '-- tracking/
'-- bin/
```

Descripción: el árbol anterior refleja la organización fuente del proyecto. `src/` contiene todo el código Java y los recursos usados en ejecución; dentro de `assets/` están las imágenes, banderas, fondos y ficheros de idioma. La carpeta `bin/` no se incluye porque solo almacena los binarios generados al compilar y no forma parte del código fuente.

Clases principales

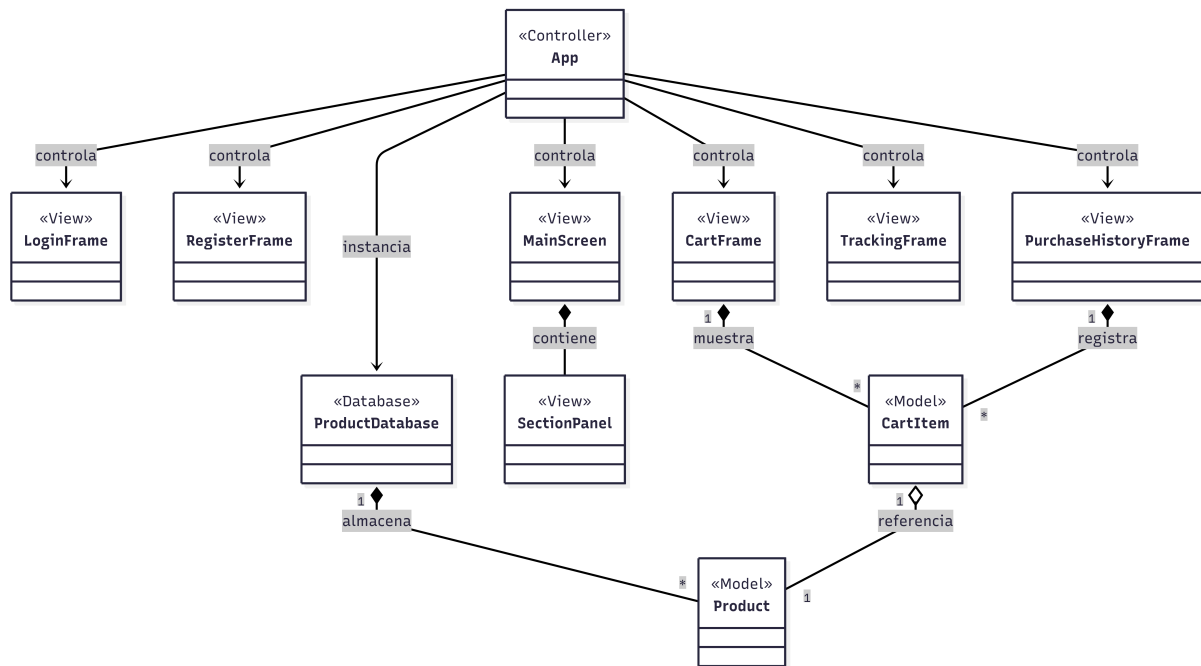


Figure 1: Diagrama de clases de la aplicación

Descripción: 'App' concentra el estado compartido, la navegación y la autenticación. 'LoginFrame' y 'RegisterFrame' gestionan el acceso y el alta de usuarios. 'MainScreen' muestra la portada con las secciones y productos destacados. 'SectionPanel' carga el catálogo de una categoría concreta y 'ProductDatabase' centraliza los arrays de productos y la creación de tarjetas visuales. 'Product' presenta el detalle de un artículo y permite añadirlo al carrito. 'CartFrame' gestiona el carrito activo, 'PurchaseHistoryFrame' muestra el historial de compras y 'TrackingFrame' representa el seguimiento del último pedido.

Otros diagrama relevante

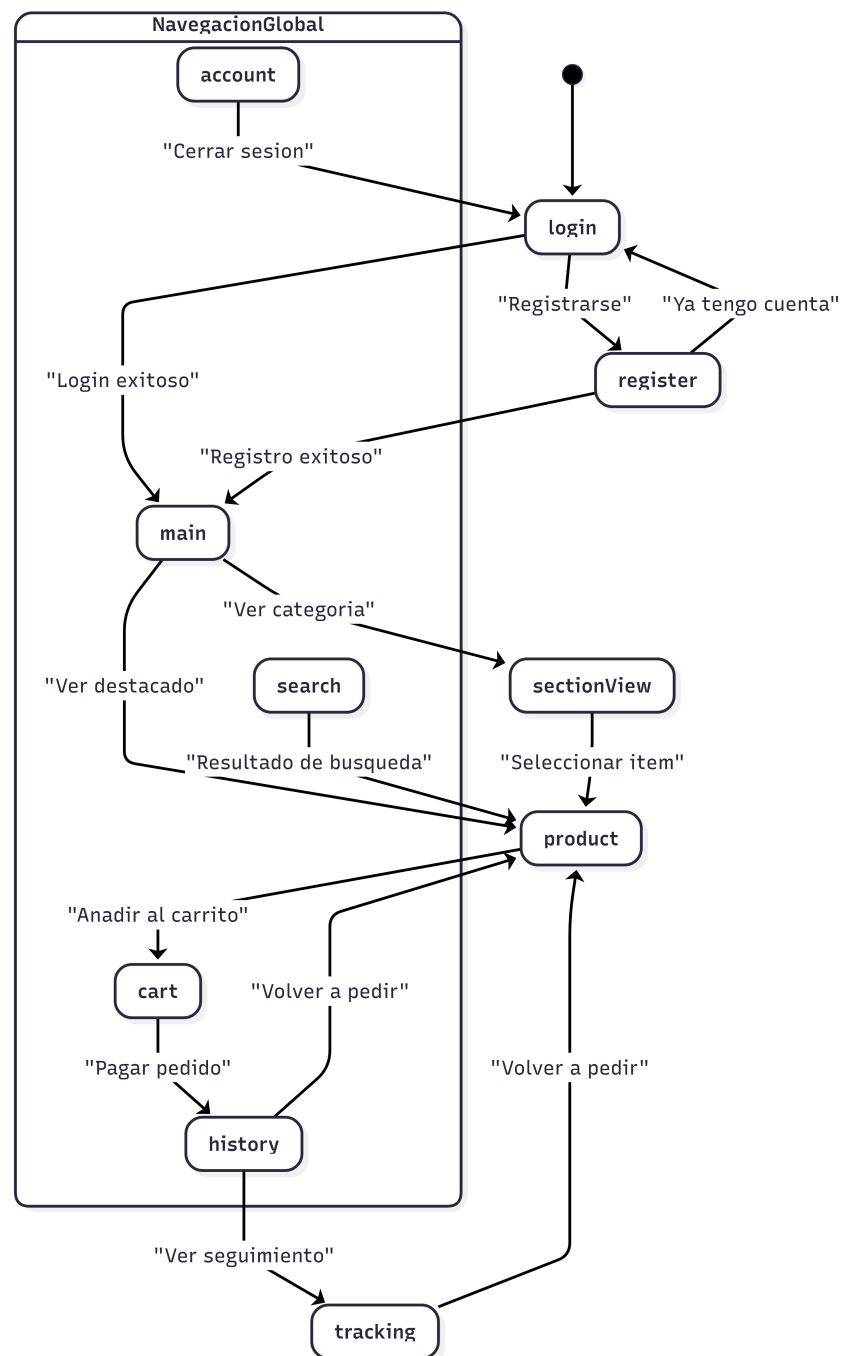


Figure 2: Diagrama de navegación entre pantallas

Descripción: el diagrama extra más útil en este proyecto es el de navegación entre pantallas. Muestra el recorrido login/registro, acceso a la pantalla principal, navegación a catálogo o producto, paso al carrito, consulta del historial y acceso al seguimiento del último pedido.

2.2 Decisiones de diseño

- **Consistencia visual:** la interfaz final mantiene una estética idéntica al prototipo original, aunque pueden existir ajustes de usabilidad o legibilidad.
- **Reutilización de componentes:** se reutilizan paneles redondeados, botones personalizados, la barra de navegación inferior y los mismos estilos tipográficos para mantener coherencia entre pantallas.
- **Separación de responsabilidades:** la interfaz queda en las clases de pantalla, los datos de producto y carrito se encapsulan en `ProductDatabase` y `CartItem`, y `App` centraliza navegación, sesión y estado compartido.

2.3 Internacionalización

La interfaz está internacionalizada en al menos dos idiomas, tal como exige el enunciado. En esta implementación se han preparado los recursos de texto para permitir el cambio entre idiomas sin modificar la estructura general de la interfaz.

- **Idiomas disponibles:** español e inglés.
- **Archivos de recursos:**
`src/assets/bundle/Bundle_es_ES.properties`
`src/assets/bundle/Bundle_en_GB.properties`
- **Mecanismo de selección:** el selector de idioma en `LoginFrame`, `RegisterFrame` y `Account` cambia el `Locale` activo y recarga el `ResourceBundle` sin rehacer la aplicación completa.
- **Textos traducidos:** se traducen etiquetas, títulos, mensajes emergentes, botones de acción y nombres de secciones, de forma que la navegación se mantiene igual en ambos idiomas.

2.4 Aspectos destacados de la implementación

La interfaz desarrollada no se limita a replicar el aspecto visual del prototipo, sino que integra soluciones estructurales para manejar el estado y la interacción en Swing de forma centralizada. A continuación se detallan los elementos técnicos más relevantes:

- **Gestión dinámica del catálogo:** la vista de productos evita la duplicación de código visual. Las tarjetas de los artículos se generan dinámicamente mediante el método genérico `createItemCard` en la clase `ProductDatabase`. Cuando el usuario abre una categoría concreta desde `MainScreen`, `SectionPanel` recibe la matriz de datos correspondiente y ensambla la vista al vuelo.
- **Motor de búsqueda unificado:** la barra de búsqueda consolida todas las categorías del inventario en tiempo real utilizando la API de flujos de Java (`Stream.of().flatMap`). Esto permite filtrar coincidencias en un único array bidimensional, devolviendo los resultados filtrados sin saturar el hilo de eventos de la interfaz.

- **Ciclo de vida del carrito y estado compartido:** los artículos seleccionados se instancian como `CartItem` y se almacenan en una colección estática dentro de la clase `App`. La interfaz del carrito no guarda datos, sino que actúa como un visor de esta lista global, calculando el precio total dinámicamente según las cantidades.
- **Persistencia en sesión para historial y seguimiento:** al procesar una compra, el flujo invoca `App.recordPurchase`. Este método transfiere los elementos del carrito activo al historial general (`purchaseHistory`), vinculándolos al identificador del usuario autenticado. El panel de seguimiento consulta esta misma estructura para aislar el último pedido realizado.
- **Internacionalización dinámica:** el cambio de idioma no exige reiniciar la aplicación. Al modificar la selección desde la vista de acceso, se actualiza el `ResourceBundle` global y la aplicación invoca el método `App.refresh`, el cual recarga las vistas en el `CardLayout` inyectando las nuevas cadenas de texto sin perder el flujo de navegación.

2.5 Relación con los conceptos teóricos

- **Visibilidad del estado del sistema y feedback inmediato:** el usuario nunca debe quedar desorientado tras ejecutar una acción. Esto se resuelve mediante el método `showAppMessage` en la clase `App`, que despliega un componente flotante efímero (*banner*) controlado por un `Timer`. Este mecanismo proporciona una confirmación inmediata al añadir un artículo al carrito o procesar un pedido, garantizando que el sistema mantenga siempre informado al usuario de lo que está ocurriendo sin bloquear el hilo principal ni recurrir a diálogos modales disruptivos.
- **Consistencia y estándares:** la consistencia interna se garantiza centralizando la estética y el comportamiento en la clase distribuidora `App`. La barra de navegación inferior (`bottomBar`) mantiene un comportamiento predecible gracias a `updateNavSelection`, que permuta los iconos ordinarios por sus versiones seleccionadas (*Selected.png*), reforzando la noción de ubicación espacial dentro del entorno. Asimismo, el uso sistemático de tipografías específicas (*Inika-Regular.ttf*) y componentes personalizados como `RoundedPanel` unifica la experiencia visual en todas las vistas de catálogo.
- **Prevención de errores:** la navegación basada en `CardLayout` no se gestiona de forma libre o descontrolada, sino mediante una pila acotada de historial (`navigationHistory` implementada con un `ArrayDeque`). Al limitar el rastro a un máximo de cinco pantallas (`MAX_NAV_HISTORY = 5`), se evita que el usuario quede atrapado en bucles infinitos al usar el botón de retroceso. Del mismo modo, el método `isTransientScreen` aísla los estados de inicio de sesión y registro, impidiendo un retroceso ilegal a pantallas de autenticación una vez que la sesión está activa y el estado global ha cambiado.
- **Flexibilidad y eficiencia de uso:** la interfaz se adapta dinámicamente al contexto del usuario. En lugar de construir paneles estáticos para cada categoría de productos, la arquitectura utiliza `ProductDatabase.createItemCard` para generar elementos de interfaz al vuelo basándose en las matrices de datos solicitadas por `SectionPanel`. Esto optimiza el rendimiento de la aplicación en memoria y reduce la carga cognitiva del usuario al presentar la información estructurada bajo un mismo patrón de interacción, independientemente de la sección consultada.

2.6 Capturas de la interfaz

Se incluyen a continuación las capturas de las pantallas principales. Cada imagen va acompañada de una breve explicación del comportamiento mostrado para que la memoria pueda leerse sin depender de las capturas.

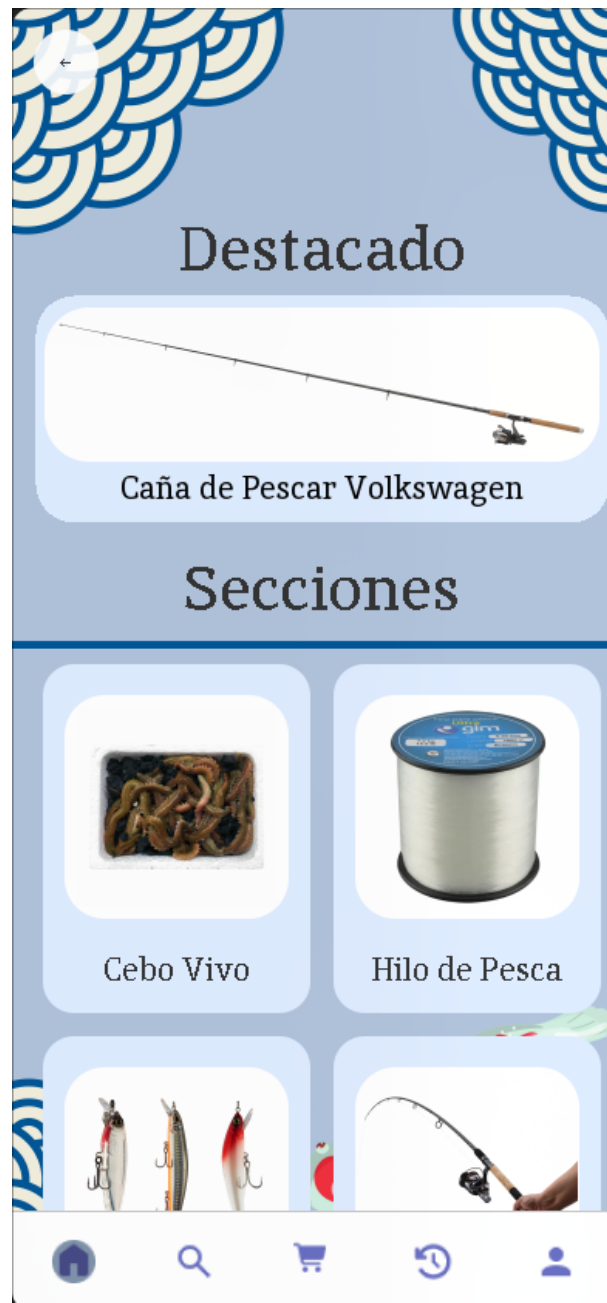


Figure 3: Pantalla principal

Descripción: vista de entrada a la aplicación con acceso a las secciones, productos destacados y navegación inferior.

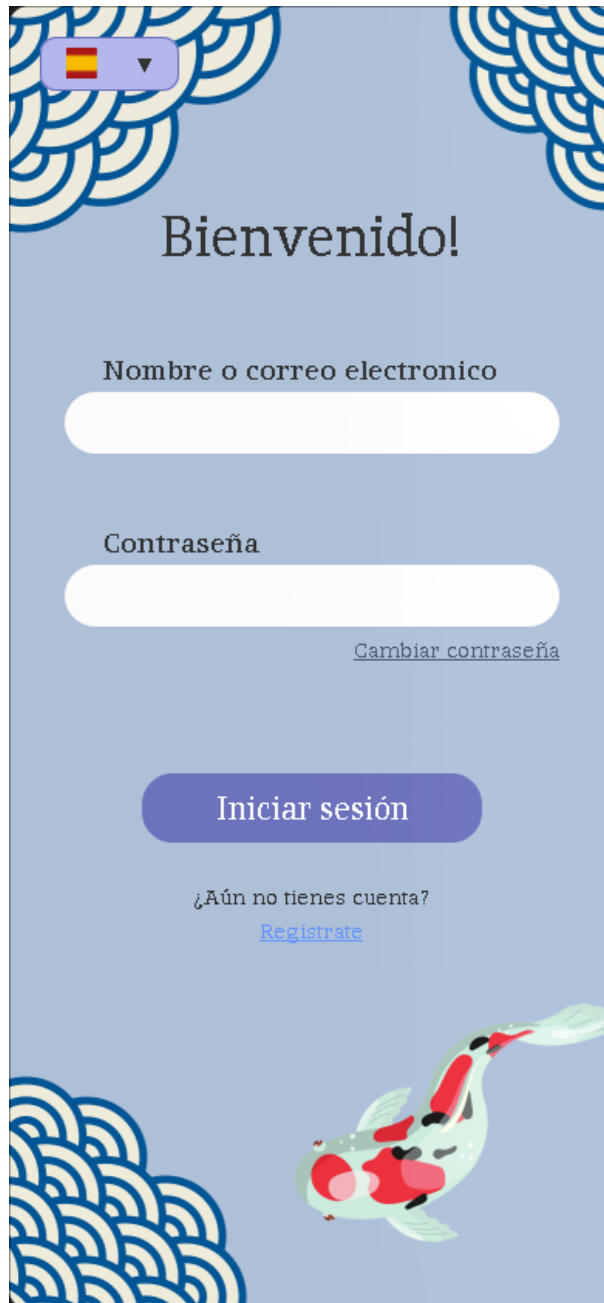
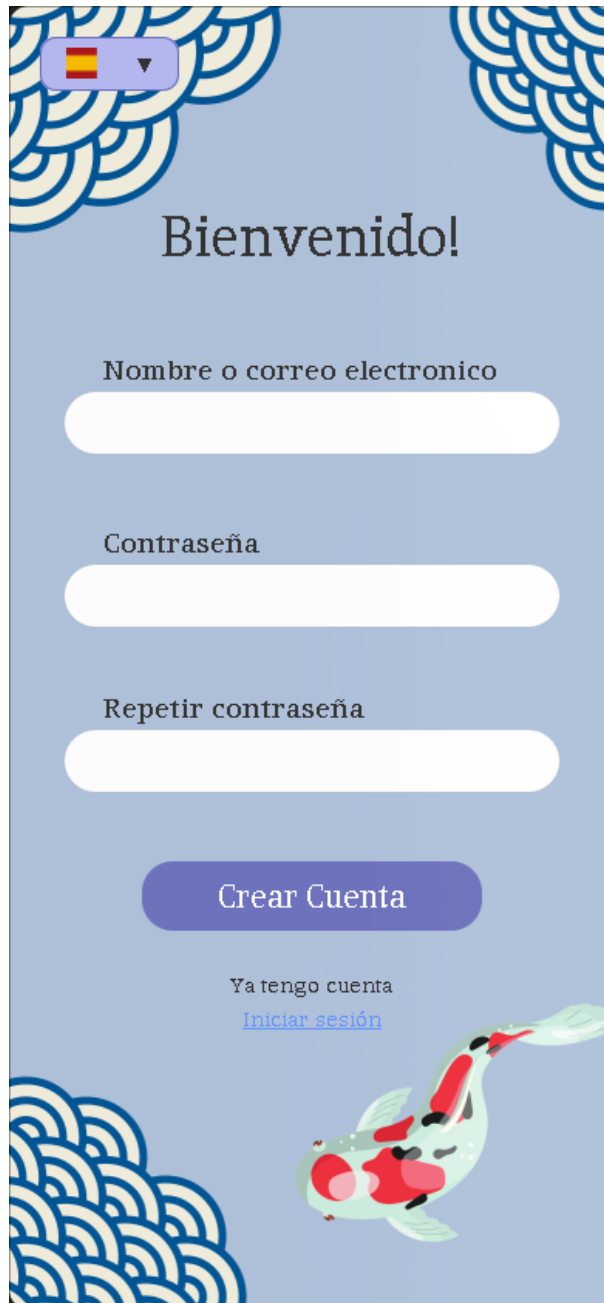


Figure 4: Pantalla de inicio de sesión

Descripción: formulario de acceso con selección de idioma, validación de credenciales y enlace al registro.



 ▼

Bienvenido!

Nombre o correo electronico

Contraseña

Repetir contraseña

Crear Cuenta

Ya tengo cuenta

[Iniciar sesión](#)

The registration screen features a light blue background with decorative white clouds in the top-left and bottom-left corners, and a colorful koi fish in the bottom-right corner. At the top, there is a language selector showing the Spanish flag and a dropdown arrow. The main heading 'Bienvenido!' is centered. Below it are three input fields: 'Nombre o correo electronico', 'Contraseña', and 'Repetir contraseña'. A purple 'Crear Cuenta' button is positioned below the input fields. At the bottom, there is a link 'Iniciar sesión' preceded by the text 'Ya tengo cuenta'.

Figure 5: Pantalla de registro

Descripción: formulario para crear una cuenta nueva con los mismos controles de idioma y acceso que la pantalla de inicio.



Figure 6: Pantalla de búsqueda o catálogo

Descripción: catálogo filtrable donde el usuario localiza productos por nombre antes de abrir su ficha.



Figure 7: Pantalla del carrito

Descripción: resumen de los productos añadidos, con ajuste de cantidades, cálculo del total y acceso al proceso de compra.



Figure 8: Pantalla de historial de compras

Descripción: listado de pedidos realizados por el usuario con los elementos de cada compra y su información básica.

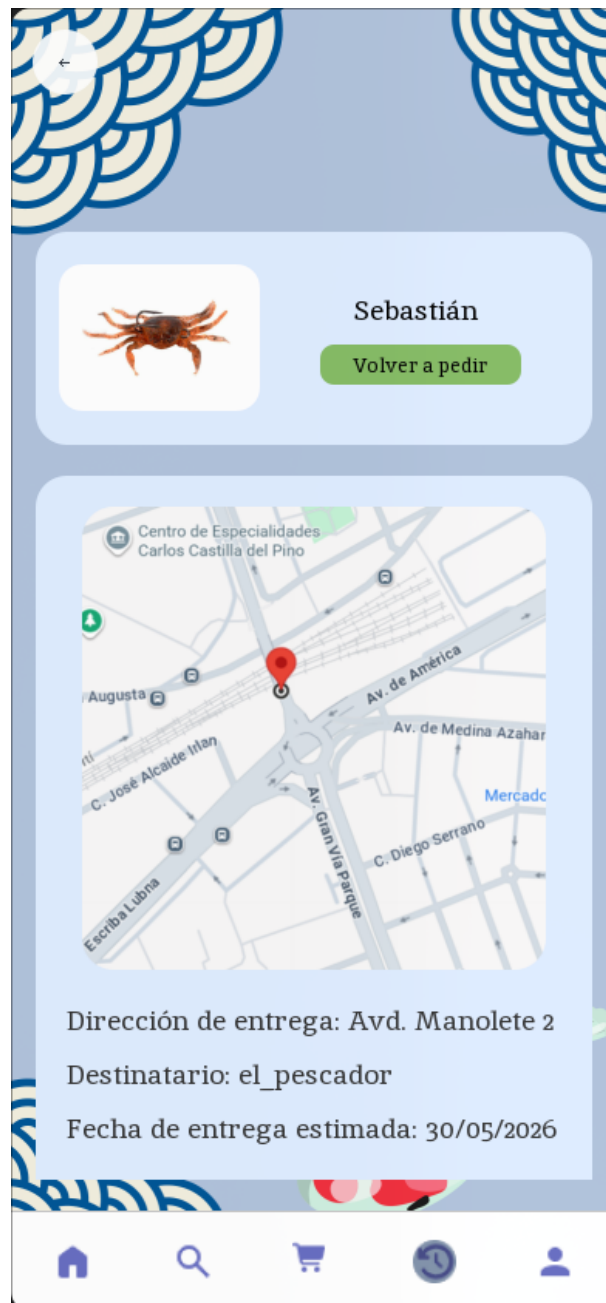


Figure 9: Pantalla de seguimiento

Descripción: vista de seguimiento del último pedido para consultar el estado de la compra y sus detalles.



Figure 10: Pantalla de cuenta

Descripción: panel de cuenta del usuario con información personal y opciones asociadas a su sesión.

3 Tutorial o breve manual de usuario

3.1 Requisitos previos

- Java 17 o superior.
- Un entorno con `javac` y `java` disponibles.
- No se requieren dependencias externas adicionales.

3.2 Compilación

Se incluye un script de construcción que compila las clases, empaqueta los recursos y genera un fichero JAR ejecutable.

1. Abrir una terminal en la raíz del proyecto.
2. Hacer ejecutable el script de construcción: `chmod +x build.sh`.
3. Ejecutar el script para compilar y empaquetar: `./build.sh`.

```
./build.sh
```

3.3 Ejecución

Para ejecutar la aplicación desde el JAR generado:

```
java -jar uco-si-final.jar
```

Si necesita ejecutar sin usar el JAR, puede compilar y ejecutar en el classpath:

```
javac -d bin src/*.java  
java -cp bin App
```

3.4 Uso de la aplicación

El recorrido del usuario se ha estructurado de manera secuencial para que resulte intuitivo, imitando la dinámica de una tienda virtual tradicional. A continuación, se detalla el flujo principal paso a paso:

1. **Autenticación y configuración:** Al arrancar el programa, la vista inicial solicita iniciar sesión con una cuenta existente (correo y contraseña) o crear una cuenta nueva desde el formulario de registro. Por defecto, la aplicación incluye una cuenta de prueba con el correo **el_pescador@gmail.com** y la contraseña **MeGustanLosPeces123**. Antes de acceder, un selector en la esquina superior permite cambiar el idioma entre español e inglés directamente desde la pantalla de acceso o registro.
2. **Navegación principal:** Una vez dentro de la sesión, el usuario dispone de una barra inferior persistente. Esta herramienta sirve para navegar de forma rápida hacia las áreas de inicio, búsqueda, carrito, historial o cuenta, independientemente de la vista actual.
3. **Exploración de productos:** En la pantalla de inicio, se puede entrar en una sección específica (como anzuelos, cañas o cebos) para explorar esa categoría. Al pulsar sobre la tarjeta de un producto, la interfaz muestra su ficha con descripción y precio, permitiendo añadirlo al carrito desde allí. Como alternativa, la sección de búsqueda filtra coincidencias de texto en todo el catálogo.
4. **Gestión y confirmación de compra:** La vista del carrito permite revisar los artículos seleccionados y ajustar cantidades con botones interactivos. Para completar la compra, el botón "Pagar" despliega un panel modal que exige introducir una dirección de envío. Al confirmar los datos, el sistema finaliza el pedido, vacía el carrito actual y guarda la información en el historial para su seguimiento posterior.

4 Conclusiones

En esta sección se deben resumir los resultados obtenidos, las dificultades encontradas y una valoración final de la solución desarrollada.

Resultados

Se ha cumplido el objetivo principal: la aplicación cuenta con una interfaz funcional, navegación entre pantallas, catálogo de productos, búsqueda, carrito, historial, seguimiento y cambio de idioma. La principal limitación es que varias partes siguen apoyándose en datos de prueba y quedan capturas o diagramas por sustituir en la memoria.

Dificultades y mejoras futuras

- La adaptación del prototipo a Swing obligó a ajustar tamaños y contenedores; se resolvió unificando paneles redondeados y layouts reutilizables.
- La internacionalización exigió centralizar textos en `ResourceBundle`; con ello se evitó duplicar la interfaz para cada idioma.
- Como mejora futura, conviene persistir usuarios y compras en disco o base de datos.
- También sería útil completar la compra con validaciones más realistas y sustituir todos los recursos provisionales.

Valoración final

El trabajo permite practicar diseño de interfaces, modularización y gestión de estado en una aplicación Swing completa. Además, deja una base clara para seguir mejorando la experiencia de usuario y la persistencia de datos.

A Anexos

Si se considera oportuno, aquí se pueden añadir tablas, notas adicionales, capturas extra, fragmentos de configuración o referencias técnicas complementarias.

A.1 Referencias de recursos

- Recursos gráficos incluidos en `src/assets/`.
- Documentación de Java Swing y de `ResourceBundle`.